

Project Architecture — Invitation Card Editor

Clean Architecture, offline-first, Flutter. No UI or business logic yet — structure only.

1. Why Clean Architecture (in plain terms)

We separate the app into three layers so that each piece has one job:

- **Presentation** — what the user sees and taps (screens, widgets).
- **Domain** — the app's core rules ("what a template is," "how a smart field works") — has zero dependency on Flutter or any package. Pure logic.
- **Data** — how things are actually stored/loaded (files, database, device storage).

Why this matters for you (a beginner): later, if we ever change *how* something is stored (e.g., swap databases) or *how* something looks (redesign a screen), we only touch one layer — the rest of the app doesn't break. It also means each piece is small and easy to explain individually.

2. Complete Folder Structure

```

lib/
├── main.dart                                # App entry
point – starts the app, sets up theme & routing
|
├── core/                                    # Shared
code used everywhere, belongs to no single feature
|   ├── constants/
|   |   ├── app_colors.dart                # The Warm
Ivory & Gold palette as reusable constants
|   |   ├── app_text_styles.dart          #
Serif/sans-serif text styles from the design spec
|   |   └── app_dimensions.dart           # Spacing,
button heights (56dp), corner radius, etc.
|   └── theme/
|       └── app_theme.dart                 # Assembles
colors + text styles into one Flutter ThemeData
|   └── errors/
|       └── failures.dart                 # Common
"something went wrong" types (e.g.,
FileLoadFailure)
|   └── utils/
|       └── ...                          # Small
reusable helper functions (date formatting, etc.)
|       └── widgets/
|           └── ...                      # Truly
generic shared widgets (e.g., PrimaryButton,
LabeledIcon)
|
├── features/                              # One folder
per app feature – mirrors the design spec's screens
|   |
|   ├── library/                          # Screen 1:
Home / My Invitations
|   |   ├── data/
|   |   |   ├── models/                  # How a
saved invitation looks in storage
|   |   |   └── repositories/            #
Reads/writes the list of saved invitations

```

```

|   |   |   |— domain/
|   |   |   |— entities/           # The
"pure" idea of an Invitation (no storage details)
|   |   |   |— usecases/           # e.g.
GetAllInvitations, DeleteInvitation,
DuplicateInvitation
|   |   |   |— presentation/
|   |   |   |— screens/           # The Home
screen itself
|   |   |   |— widgets/           # Thumbnail
card, empty state, etc.
|   |   |   |— source_selection/   # Screen 2:
New Invitation (Upload / Template / Blank)
|   |   |   |— presentation/
|   |   |   |— screens/
|   |   |   |— widgets/
|   |   |   |— templates/         # Screen 3:
Template Library + "Save as Template" (Screen 5)
|   |   |   |— data/
|   |   |   |— domain/
|   |   |   |— usecases/           # e.g.
SaveTemplate, GetTemplates, ApplySmartField
|   |   |   |— presentation/
|   |   |   |— editor/             # Screen 4:
The Editor (the core feature)
|   |   |   |— data/
|   |   |   |— models/             #
TextBoxModel: position, font, size, color, rotation
|   |   |   |— repositories/
|   |   |   |— domain/
|   |   |   |— entities/           #
TextElement, Canvas, SmartField
|   |   |   |— usecases/           # AddText,
MoveText, FormatText, RecordSmartField,
ApplyVoiceInput
|   |   |   |— presentation/

```

```

|   |   |   | screens/           #
editor_screen.dart
|   |   |   | widgets/           #
canvas_view, formatting_panel, text_box_widget,
voice_input_button
|   |   |   |
|   |   |   | smart_font_matching/   # Feature
6.2: analyzes uploaded image, suggests
font/size/color
|   |   |   | data/
|   |   |   | domain/
|   |   |   | usecases/           #
AnalyzeInvitationStyle
|   |   |   | presentation/
|   |   |   | widgets/           # The
"Reading your invitation..." indicator
|   |   |   |
|   |   |   | export/             # Screen 6:
Save as PDF / Print
|   |   |   | domain/
|   |   |   | usecases/           #
ExportToPdf, PrintInvitation
|   |   |   | presentation/
|   |   |   |
|   |   |   | settings/           # Screen 7:
Settings
|   |   |   | presentation/
|   |   |   |
|   |   |   | routes/
|   |   |   | app_router.dart      # Central
place listing every screen and how to navigate
between them
|   |   |   |
|   |   |   | injection/
|   |   |   | service_locator.dart # Wires
everything together (explained in Step 2)

test/                               # Mirrors
the lib/ structure – one test folder per feature

```

(later step)

```
assets/  
├── fonts/                                # Font files  
used in the app and offered to users  
├── images/                              # Onboarding  
illustrations, icons, empty-state art  
└── templates/                          # Bundled  
starter templates (wedding, birthday, etc.)
```

Rule of thumb for how we'll work: each feature folder is self-contained. When we build "the Editor" in a later step, we'll only be working inside `features/editor/` (plus a few shared `core/` pieces) — not touching six files scattered across the project.

3. Dependencies (packages) and why each is needed

Package	Purpose	Why we need it
flutter_riverpod	State management	Connects your taps/typing to what's shown on screen, cleanly, without messy shared variables. Works well with clean architecture's layer separation.
go_router	Navigation	Manages moving between screens

Package	Purpose	Why we need it
		(Home → Editor → Export) in one central, predictable place instead of scattered logic.
get_it	Dependency injection	Wires the layers together (e.g., connects the Editor screen to its use-cases) without screens needing to know storage details.
hive + hive_flutter	Local offline database	Stores your saved invitations, templates, and settings entirely on-device — no internet needed, fast, simple key-value storage.
path_provider	File system paths	Finds the correct, safe folder on the Android device to store files (invitations, exported PDFs).
file_picker	File selection	Lets the user pick a PDF/JPG/PNG

Package	Purpose	Why we need it
		from their device storage when uploading an invitation (Screen 2).
pdf	PDF generation	Builds the final exported invitation as a real PDF file (Screen 6, "Save as PDF").
printing	Printing & PDF preview	Hands off to Android's native print dialog and can render PDF pages for preview — offline-compatible.
image	Image processing	Reads and manipulates uploaded JPG/PNG files (for placing them on the canvas and for Smart Font Matching analysis).
pdf_render or syncfusion_flutter_pdfviewer	Rendering uploaded	Uploaded invitations that are PDFs need to

Package	Purpose	Why we need it
<i>(one, to be decided in the editor step)</i>	PDFs on-screen	be displayed as an image on the editing canvas — this converts PDF pages to a viewable/editable surface.
speech_to_text	Voice typing	Powers the "Voice Type" feature — converts the user's spoken words into text, fully on-device where supported.
google_fonts <i>(bundled offline, not fetched at runtime)</i> or local font assets	Font variety	Provides the elegant serif/sans-serif fonts from the design system, plus the font list users choose from in the formatting panel.
flutter_colorpicker	Color selection	Powers the "More Colors" option beyond the curated swatch palette in the formatting panel.

Package	Purpose	Why we need it
share_plus	Sharing exported files	Powers the "Share" button after export (Screen 6), using Android's native share sheet.
equatable	Value comparison	Small utility that makes domain entities (like TextElement) easy to compare — keeps clean architecture code simple and bug-free.

Important offline note: every package above works fully offline once installed — `speech_to_text` uses the on-device recognizer, fonts are bundled as assets rather than downloaded, and there's no analytics, ad, or networking package anywhere in this list, in line with the "100% offline" requirement.

4. What's intentionally NOT here yet

No widgets, no screens' actual content, no state logic, no theme colors applied — just the skeleton and its labeled

rooms. This gets built one feature folder at a time, starting wherever you'd like once you confirm this structure.